03-Git_GitHub Troubleshooting Python Introduction and Class Feedback

Git and GitHub Setup and Troubleshooting

- Many students struggled with pushing code from their local system to GitHub repositories.
- Instructor guided on creating and initializing directories, using commands (e.g., `git init`, `git add .`, `ls -al`, `git status`, `git commit -m`, `git push`).
- Emphasis on using Git Bash (especially for Windows), understanding folder paths, and switching directories.
- Common confusion resolved between pushing (code to GitHub) vs. pulling/cloning (downloading from GitHub).
- Detailed demonstration on specifying files to add (e.g., `git add .`, `git add f1.py`, `git add *.py`), with explanations on when to use each.

Authentication and Security (Personal Access Token)

- Students faced authentication errors due to recent GitHub policy changes disallowing password authentication; personal access tokens are now required.
- Instructor walked students through generating a personal access token (Settings > Developer Settings > Personal Access Tokens) and explained scope selection.
- For Windows, instructed on updating credentials using the system's Credential Manager by replacing passwords with the generated token.
- Clarified that a token with 'no expiration' can be used to avoid repeated generation.

Git Branch Structure and Changes

- Explained the transition from 'master' to 'main' as the default branch in GitHub; provided command (`git branch -m main`) to switch branches.
- Clarified that both branches functionally serve the same primary role in repositories.
- Discussed scenario of having both 'main' and 'master' branches and assured no cause for concern.

Workflow and Practical Git Usage

- Common issues encountered: push errors, credential/authentication pop-ups, missing `.git` folder, mismatch between local and remote branches.
- Used Vim editor (`vim f1.py`, insert/escape/save commands) for live code editing and committing changes.
- Stressed that, once initial setup is complete, only a few Git commands (`add`, `commit`, `push`) are routinely needed for workflow.

Introduction to Python and Data Types

- Brief start to Python lessons, focusing on data types.
- Fundamental data types: int, float, complex, boolean, none, string.
- Derived data types: list, tuple, set, frozen set, dictionary, bytes, byte array, array.
- Quick quiz to gauge participants' backgrounds and comfort with coding.

Jupyter Notebook Shortcuts

- Shared essential Jupyter shortcuts: copy (C), paste (V), cut (X), delete (DD), shift+enter (run), add cell above (A), add cell below (B), code/markdown toggling (M), and the importance of practice for retention.

Class Feedback and Next Steps

- Widespread student feedback: challenges due to non-technical backgrounds, mismatch between instructor OS (Mac/Linux) and majority using Windows, desire for more theoretical context before hands-on practice, and requests for clearer explanations and pacing.

- Students requested a trainer with proven experience in foundational explanation and engagement.

- Decision made to switch the trainer beginning the next class, with either the new instructor or the highly-regarded Afsan (depending on availability).

- Agreed to postpone further Git/GitHub troubleshooting to dedicated weekend sessions, and continue Python lessons in weekday classes.

- Meeting and class links will remain the same, and students were advised on support channels.

Key Takeaways

- Git/GitHub basics and authentication require careful step-by-step practice, especially with recent changes.

- Use of command line and understanding folder navigation is essential for successful code management.

- Non-technical participants benefit strongly from theoretical context and aligned teaching environments (e.g., OS parity).

- Class structure will be adapted in response to feedback, starting with a new trainer and tailored troubleshooting support.